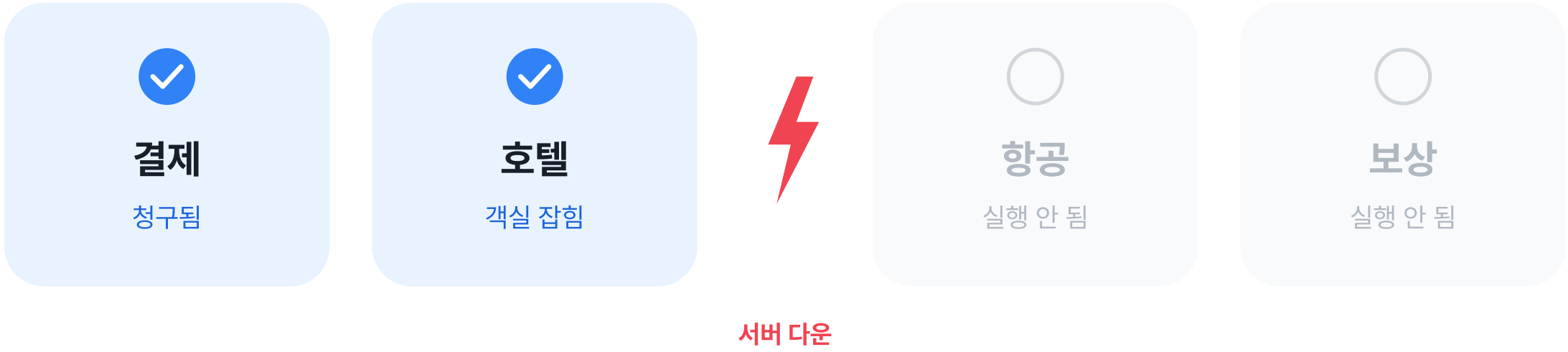
MSA 다음 트렌드는 무엇일까요?

SAGA를 직접 짜던 코드가 런타임으로 옮겨 가요

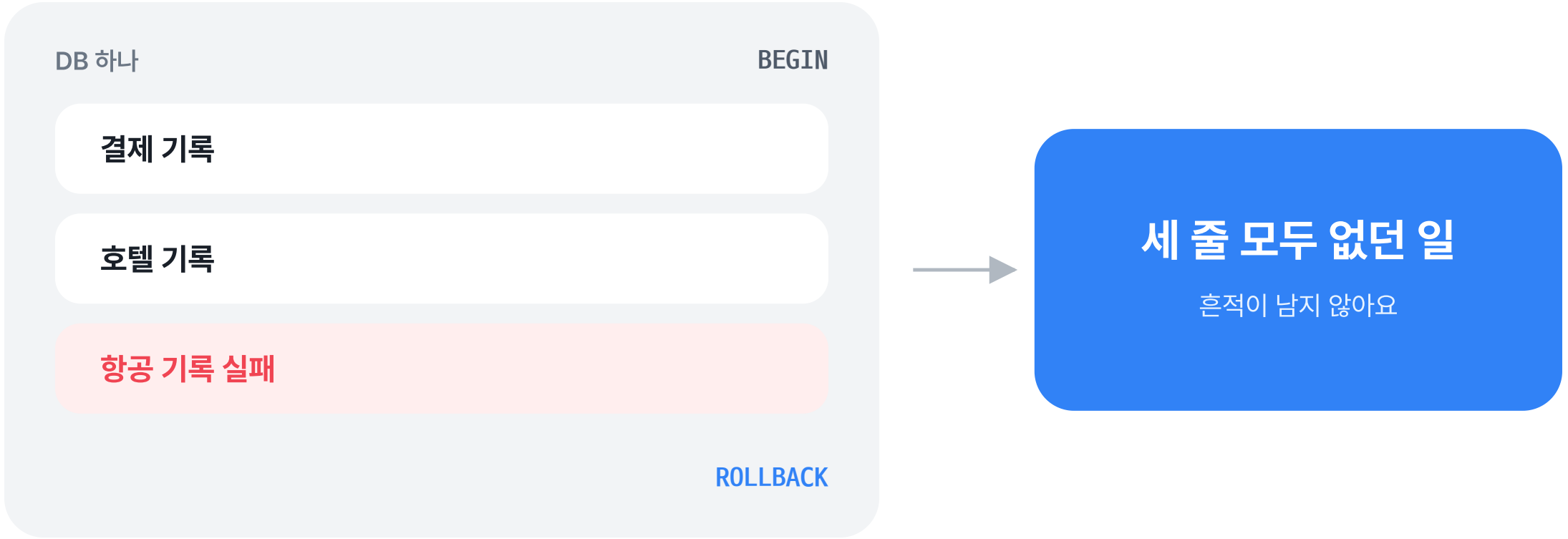
호텔까지 잡았는데 서버가 죽었어요

결제는 됐는데 항공권은 없는 예약이 남아요



DB가 하나였다면 롤백 한 번이면 끝나요

트랜잭션은 전부 되거나, 전부 안 되게 해 줘요



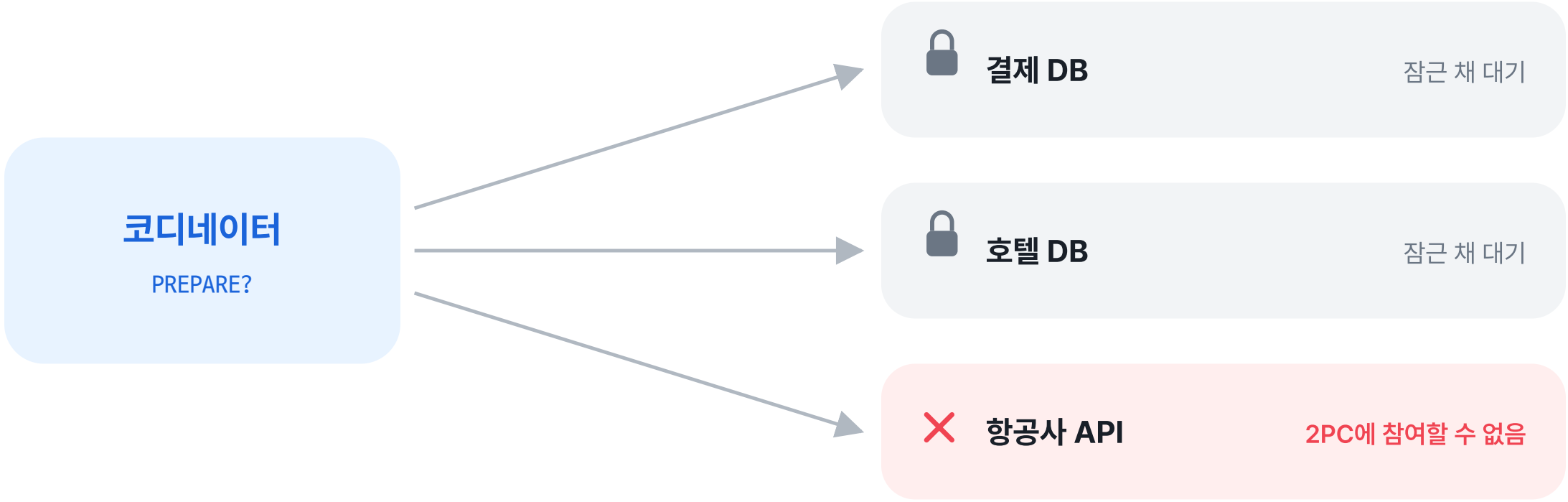
MSA에서는 트랜잭션 하나로 묶을 수 없어요

DB는 서비스마다 나뉘고, 진짜 예약은 외부 회사 시스템에 있어요



2PC로 묶으면 되지 않을까요?

외부 API는 참여할 수 없고, 하나라도 늦으면 모두 잠긴 채 기다려요



그래서 사가를 써요

단계마다 바로 커밋하고, 실패하면 보상으로 되돌려요



사가를 실행하는 방법은 두 가지예요

이벤트로 이어 가거나, 한 곳에서 지휘하거나

코레오그래피

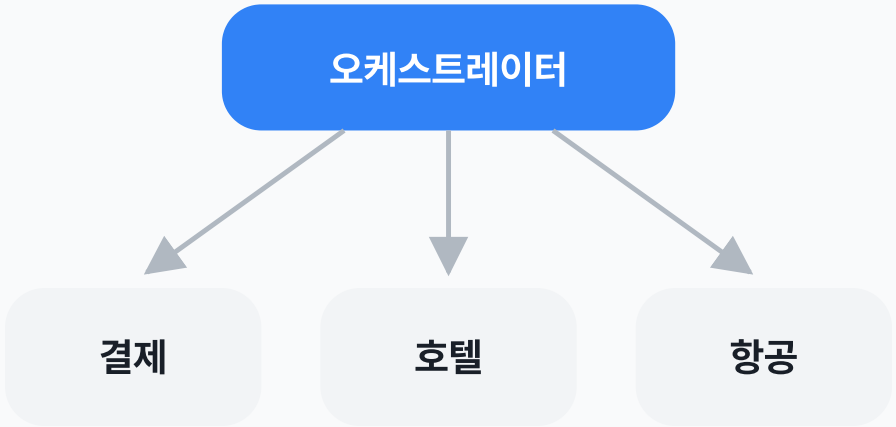
이벤트를 받고 각자 다음 일을 해요



흐름이 어디에도 한곳에 적혀 있지 않아요

오케스트레이션

한 곳에서 순서를 지휘해요



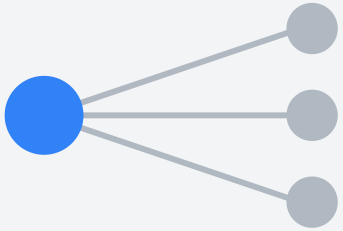
지휘자가 죽으면, 어디까지 했는지가 문제예요

이벤트로 이어 가는 방식이 많이 쓰였어요

당시에는 합리적인 선택이었어요



판단은 서비스가,
통로는 전달만 해요



발행자는 구독자를
몰라도 돼요



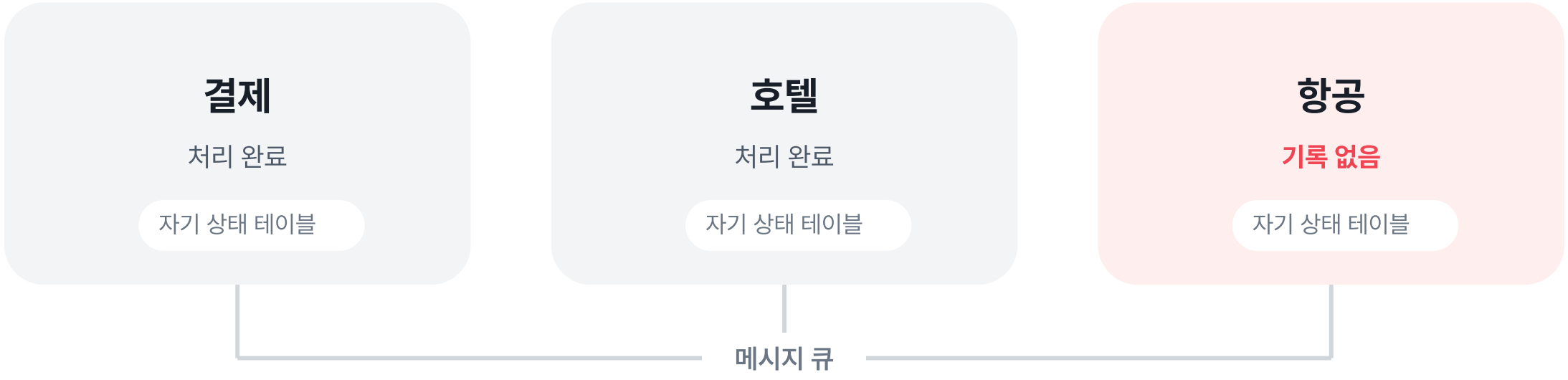
다른 이유로
이미 깔려 있었어요



대안은 흐름을
JSON으로 그렸어요

그 대신 흐름이 서비스마다 흩어졌어요

멈춘 위치를 알려면 세 곳을 이어 붙여야 해요



직접 지휘하려면, 로직보다 배관이 더 컸어요

지휘자가 죽어도 이어 가려고 서비스마다 만든 것들이에요

예약 서비스 코드

상태 전이

재시도 · 타임아웃

폴링 · 타이머

결제 → 호텔 → 항공

보상 흐름

아웃박스 발행

멱등성 검사

따로 운영하는 것

상태 테이블

아웃박스 테이블

복구 스케줄러

멱등성 키

끝난 단계는 건너뛰고, 빈 단계부터 실행해요

지속 실행은 단계마다 기록해 두고 재생해요



배관은 런타임으로 가고, 서비스엔 순서만 남아요

앞 장과 같은 자리인데, 코드가 이만큼 줄었어요

예약 서비스 코드

결제 → 호텔 → 항공

멱등성 키 전달

따로 운영하는 것

지속 실행 엔진

실행 기록 · 재시도 · 타이머 · 재배정

멱등성 키

사가가 평범한 함수가 돼요

위에서 아래로 쓰고, 보상은 except로 처리해요

```python
@workflow.defn
class BookTrip:
    @workflow.run
    async def run(self, order):
        pay   = await execute_activity(charge, order)
        hotel = await execute_activity(confirm_hotel, order)
        try:
            return await execute_activity(confirm_flight, order)
        except ActivityError:
            await execute_activity(cancel_hotel, hotel)
            await execute_activity(refund, pay)
            raise
```

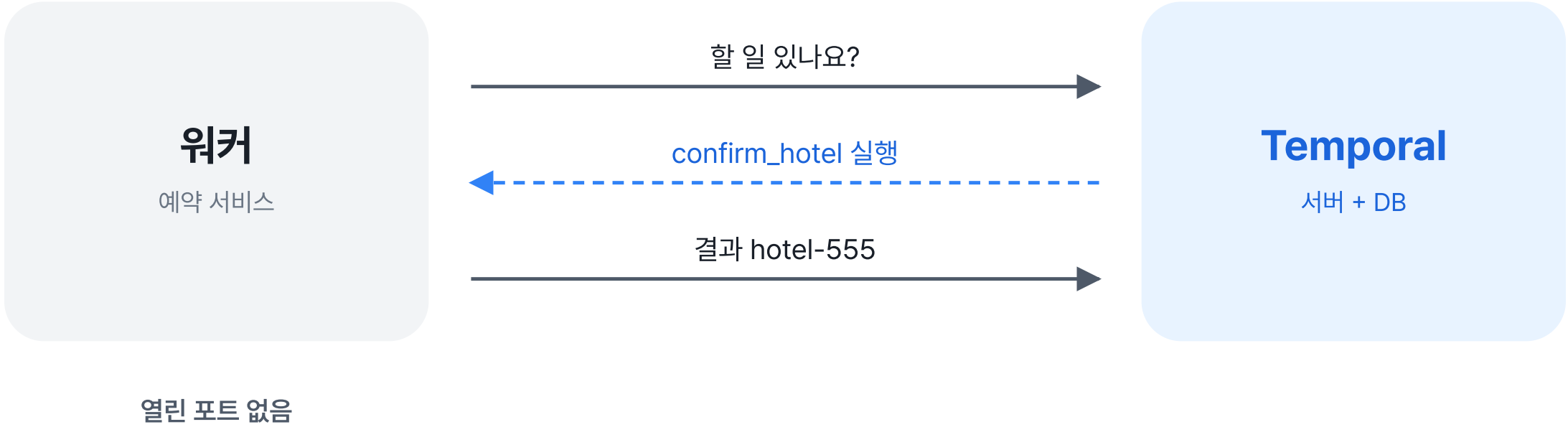
Temporal Python SDK · import와 타임아웃 옵션은 생략했어요

호출 하나가 기록 하나예요

보상은 예외 처리로 해요

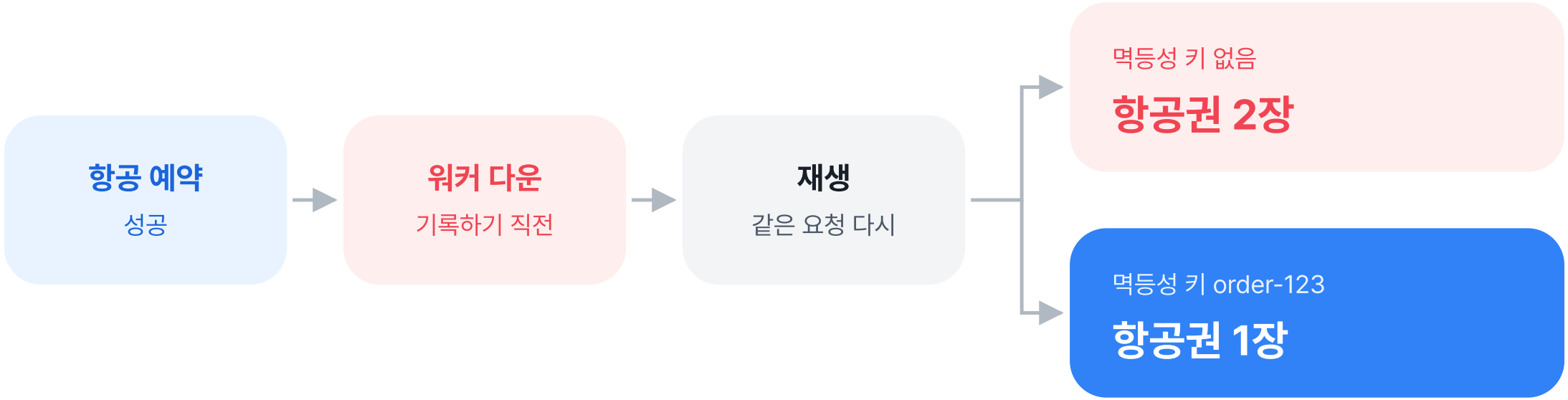
엔진이 부르지 않고, 워커가 일을 가져가요

그래서 워커는 포트를 열 필요가 없어요



멱등성은 여전히 개발자가 챙겨야 해요

기록하기 직전에 죽으면, 같은 호출이 한 번 더 나가요



대신 지켜야 할 규칙이 생겨요

코드를 재생하는 방식이라 생기는 규칙이에요

결정성

`workflow.now()`

재생해도 값이 같아야 해요

버전 관리

`workflow.patched()`

옛 경로를 남겨 둬야 해요

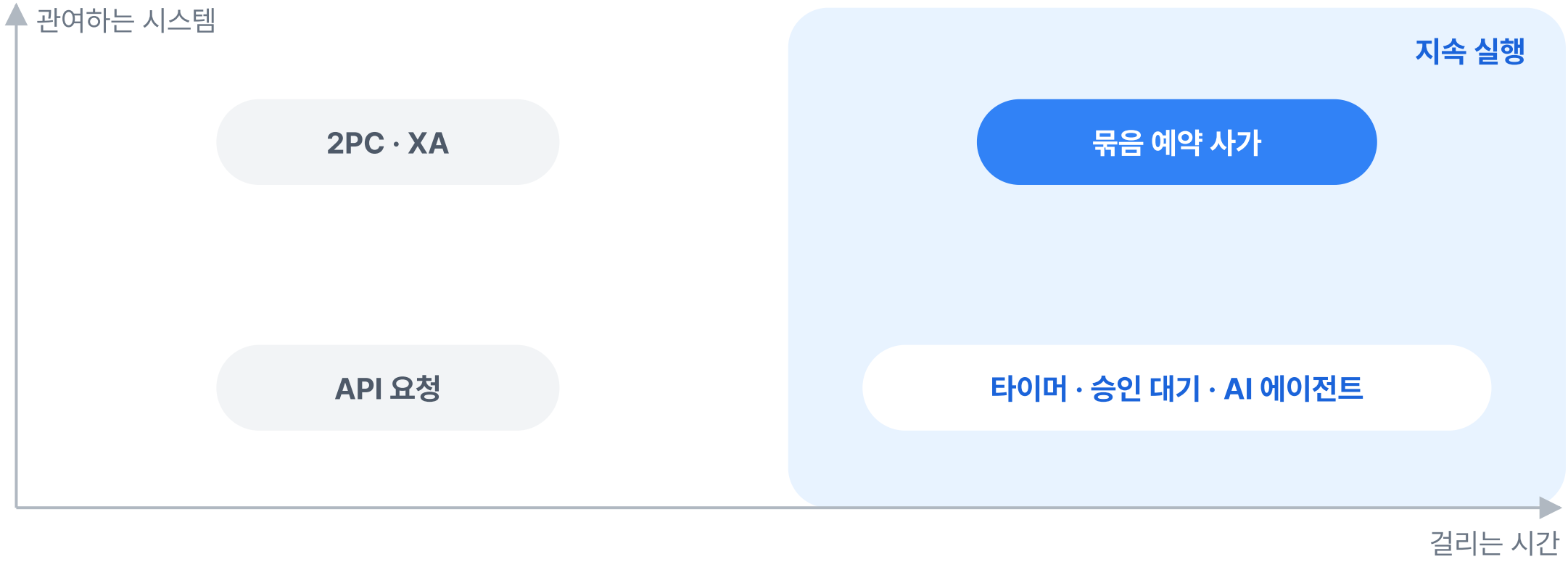
기록 한도

`continue_as_new()`

51,200 이벤트 전에 이어 가요

사가보다 넓은 문제예요

되돌릴 게 없어도, 오래 걸리는 작업이면 같은 문제가 생겨요

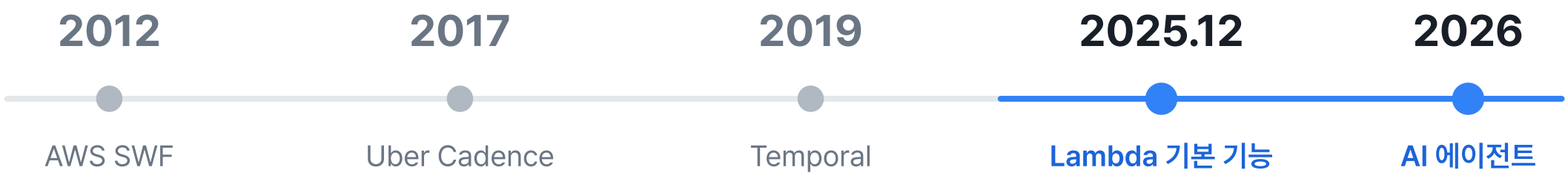


다른 계층에서도 같은 이동이 일어나요

서비스가 하던 공통 책임이 런타임으로 옮겨 가요

재시도 · 상태 · 보상	→	**지속 실행 엔진** — Temporal · Lambda durable functions
인증 · 라우팅	→	**서비스 메시 · Gateway API** — Istio ambient
에이전트 신원 · 권한	→	**에이전트 게이트웨이** — MCP · A2A

기술은 2012년부터 있었고, 확산은 지금이에요



바뀌는 건 쪼개는 방식이 아니라
책임이 놓이는 자리예요